LAPORAN HYBRID COMPUTING FOR REAL-WORLD SIMULATION AND DATA PROCESSING

Laporan ini disusun untuk memenuhi Ujian Tengah Semester
Mata Kuliah Komputasi Paralel & Terdistribusi



Disusun Oleh:
Muhammad Zaki Zaidan Binawan
247006111134

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS SILIWANGI
2026

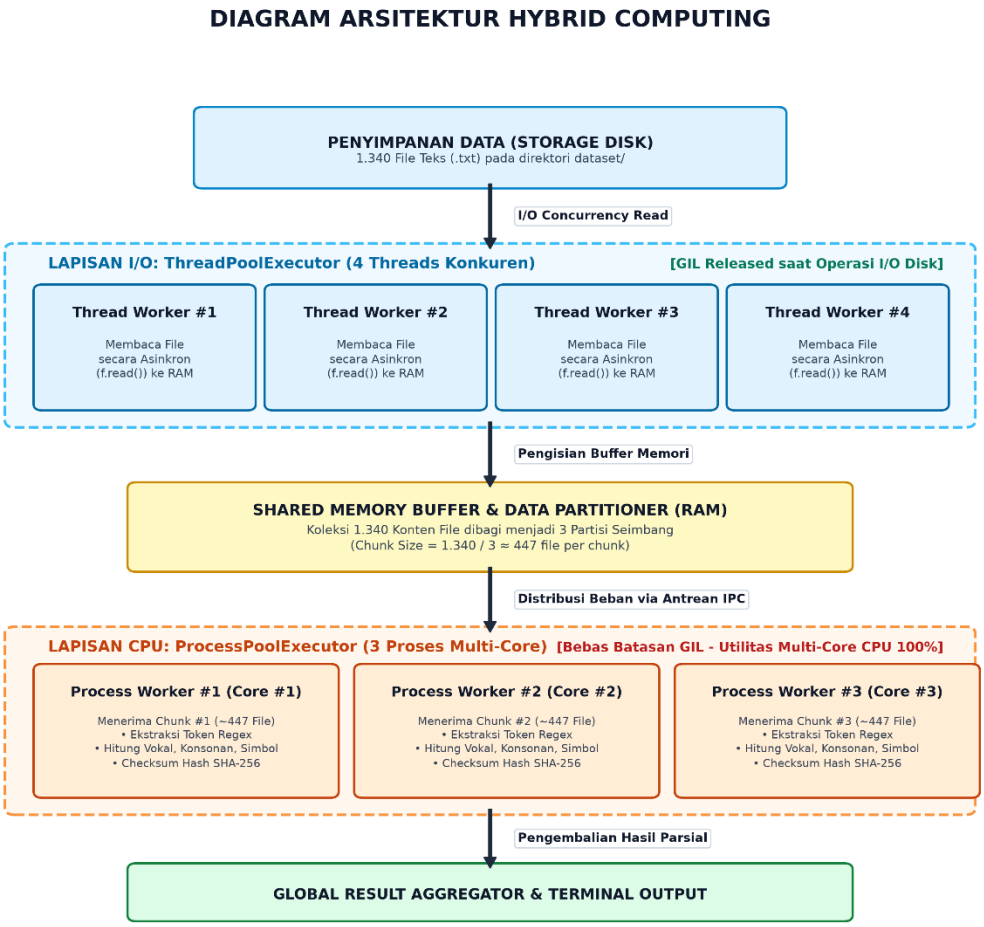
DAFTAR ISI

DAFTAR ISI 2
PARAMETER 3
BAGIAN A: Konsep & Desain 3
BAGIAN B: Implementasi Kode 5
BAGIAN C: Hasil Eksperimen 19
BAGIAN D: Analisis dan Kesimpulan 21

PARAMETER

1. **Nomor Acak Pribadi**: random.seed(247006111134).
2. **Jumlah Thread:** Dua digit terakhir NIM Mod 4 + 2 = (34 Mod 4) + 2 = 2 + 2 = **4 Thread**.
3. **Jumlah Process:** Dua digit Tengah NIM Mod 3 + 2 = (61 Mod 3) + 2 = 1 + 2 = **3 Process**.
4. **Jumlah Data:** Tiga digit terakhir NIM x 10 = 134 x 10 = **1340 file**.

BAGIAN A: Konsep & Desain



Proyek ini adalah Parallel File Analyzer. Sistem akan memproses 1.340 file teks korpus secara cepat menggunakan kombinasi dua paradigma paralelisme: Task Parallelism (Multithreading) pada lapisan I/O dan Data Parallelism (Multiprocessing) pada lapisan komputasi CPU.

- Lapsisan Input: Penyimpanan Data (Storage Disk)
  Ada 1340 file teks (file_0001.txt sampai file_1340.txt) hasil pembangkitan deterministik berbasis seed NIM (247006111134).

- Lapisan I/O: ThreadPoolExecutor (4 Threads)
  Menangani operasi I/O-Bound (membaca isi file dari disk ke dalam RAM). Pada interpreter CPython standar, ada Global Interpreter Lock (GIL) yang membatasi eksekusi bytecode hanya pada satu thread dalam satu waktu. Namun, kketika sebuah thread menjalankan ooperasi I/O (seperti membuka file, f.read()), python secara otomatis melepaskan GIL.

- Lapisan Perantara: Shared Memory Buffer & Data Partitioner (RAM)
  1340 objek tkes yang telah dibaca disimpan di RAM dan dipartisi menjadi 3 bagian seimbang (chunks): Chunk Size = (1340/3) ≈ 447 file per chunk. Pemecahan menjadi blok besar (chunking) ini bertujuan untuk meminimalkan overhead komunikasi antar proses (IPC).

- Lapisan CPU: ProcessPoolExecutor (3 Proses Worker)
  Menangani operasi CPU-Bound (komputasi intensif analisis data teks). Setiap proses worker berjalan pada inti CPU terpisah (Core #1, Core #2, Core #3) dan memproses satu chunk data (~447 file) secara independent:
  1) Tokenisasi Regex: Memecah kata menggunakan ekspresi reguler \b\w+\b.
  2) Klasifikasi Karakter: Menghitung frekuensi huruf vocal, konsonan, angka, dan symbol.
  3) Integritas & Workload: Menghitung checksum kriptografis SHA-256 untuk memverifikasi keutuhan berkas.

  Karerna analisis teks menguras siklus clock CPU, eksekusi dalam thread biasa akan terkena Batasan GIL dan berjalan lambat seperti serial. Dengan ProcessPool Executor, setiap proses memiliki ruang memori dan interpreter Python sendiri, sehingga mampu memanfaatkan multi-core CPU secara penuh 100%.

- Lapisan Agregasi & Keluaran Terminal
  Masing-masing sub-proses mengirimkan statistic parsialnya Kembali ke proses utama melalui pipa IPC (inter-Process Communication). Proses utama menjumlahkan total keseluruhan (reduction), menghitung metrik kinerja (speedup, Efisiensi, dan Throughput), serta mencetak keluaran resmi ke terminal.

BAGIAN B: Implementasi Kode

❖ Kode (generate_database.py) untuk generate dataset file teks

```
# Generator Dataset File Teks

#================================================
# Nama  : Muhammad Zaki Zaidan Binawan
# NPM   : 247006111134
# Kelas : E
#================================================

import os
import random

# Parameter Khusus NIM
NIM = "247006111134"
TOTAL_FILES = 1340  # Tiga digit terakhir NIM (134) * 10
DATASET_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")

# Kosakata sampel untuk membentuk teks realistis
SAMPLE_WORDS = [
    "komputasi", "paralel", "terdistribusi", "prosesor", "thread", "multiprocessing",
    "kinerja", "algoritma", "arsitektur", "memori", "jaringan", "skalabilitas",
    "sinkronisasi", "throughput", "latency", "optimasi", "eksekusi", "frekuensi",
    "pipeline", "concurrency", "distributed", "system", "cluster", "performance",
    "amdahl", "speedup", "efisiensi", "analisis", "dataset", "workload", "simulasi"
]

SAMPLE_TEMPLATES = [
    "Sistem {word1} modern memerlukan {word2} yang efisien untuk mencapai {word3} optimal.",
    "Dalam lingkungan {word1}, penanganan {word2} dan {word3} sangat krusial bagi sistem.",
    "Peningkatan {word1} berbanding lurus dengan alokasi {word2} pada beban {word3}.",
    "Model {word1} membagi komputasi menjadi beberapa unit {word2} guna mempercepat {word3}.",
    "Evaluasi metrik {word1} menunjukkan bahwa {word2} mampu mereduksi {word3} secara
signifikan."
]

def generate_content(rng, num_paragraphs=6):
    paragraphs = []
    for _ in range(num_paragraphs):
        sentences = []
        for _ in range(rng.randint(4, 8)):
            template = rng.choice(SAMPLE_TEMPLATES)
            w1 = rng.choice(SAMPLE_WORDS)
            w2 = rng.choice(SAMPLE_WORDS)
            w3 = rng.choice(SAMPLE_WORDS)
```

```
            sentence = template.format(word1=w1, word2=w2, word3=w3)
            # Tambahkan variasi angka dan karakter simbol secara deterministik
            if rng.random() > 0.4:
                num = rng.randint(10, 9999)
                sentence += f" (ID-{num} val: {rng.random():.4f}#@!)"
            sentences.append(sentence)
        paragraphs.append(" ".join(sentences))
    return "\n\n".join(paragraphs)

def main():
    print(f"=== GENERATOR DATASET FILE TEKS ===")
    print(f"NIM: {NIM}")
    print(f"Jumlah file target: {TOTAL_FILES}")
    print(f"Lokasi folder: {DATASET_DIR}")

    # Set random seed sesuai ketentuan soal
    random.seed(int(NIM))

    os.makedirs(DATASET_DIR, exist_ok=True)

    print("Sedang membuat file teks...")
    for i in range(1, TOTAL_FILES + 1):
        filename = f"file_{i:04d}.txt"
        filepath = os.path.join(DATASET_DIR, filename)
        content = generate_content(random, num_paragraphs=random.randint(5, 8))
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(content)

        if i % 200 == 0 or i == TOTAL_FILES:
            print(f"Progress: {i}/{TOTAL_FILES} file berhasil dibuat...")

    print(f"Selesai! Berhasil membuat {TOTAL_FILES} file teks di '{DATASET_DIR}'.")

if __name__ == "__main__":
    main()
```

Hasil Eksekusi:

```
PS Z:\4.College\2.Koding\S5\KPT\UTS_KPT> python generate_dataset.py
=== GENERATOR DATASET FILE TEKS ===
NIM: 247006111134
Jumlah file target: 1340
Lokasi folder: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\dataset
Sedang membuat file teks...
Progress: 200/1340 file berhasil dibuat...
Progress: 400/1340 file berhasil dibuat...
Progress: 600/1340 file berhasil dibuat...
Progress: 800/1340 file berhasil dibuat...
Progress: 1000/1340 file berhasil dibuat...
Progress: 1200/1340 file berhasil dibuat...
Progress: 1340/1340 file berhasil dibuat...
Selesai! Berhasil membuat 1340 file teks di 'Z:\4.College\2.Koding\S5\KPT\UTS_KPT\dataset'.
PS Z:\4.College\2.Koding\S5\KPT\UTS_KPT>
```

❖ Kode Utama (hybrid_analyzer.py)

```python
# Program Utama Proyek Hybrid Computing - Parallel File Analyzer

#================================================
# Nama  : Muhammad Zaki Zaidan Binawan
# NPM   : 247006111134
# Kelas : E
#================================================

import os
import sys
import time
import math
import re
import hashlib
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor

# Hitungan Parameter:
# - Dua digit terakhir (34) mod 4 + 2 = 2 + 2 = 4
DEFAULT_THREADS = (34 % 4) + 2

# - Dua digit tengah (61) mod 3 + 2 = 1 + 2 = 3
DEFAULT  PROCESSES = (61 % 3) + 2

# - Tiga digit terakhir (134) * 10 = 1340
DEFAULT_DATA_COUNT = 134 * 10

DATASET_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "dataset")

# TUGAS KOMPUTASI CPU-BOUND (Dijalankan oleh ProcessPoolExecutor)
```

```python
def cpu_task_analyze_chunk(file_data_list):
    """
    Fungsi CPU-bound untuk memproses sekumpulan teks file.
    Dijalankan di tingkat prosesor terpisah agar terbebas dari GIL Python.
    """
    vowel_pattern = re.compile(r'[aeiouAEIOU]')
    consonant_pattern = re.compile(r'[bcdfghjklmnpqrstvwxyzBCDFGHJKLMNPQRSTVWXYZ]')
    digit_pattern = re.compile(r'[0-9]')
    symbol_pattern = re.compile(r'[^a-zA-Z0-9\s]')
    word_pattern = re.compile(r'\b\w+\b')

    chunk_total_words = 0
    chunk_total_vowels = 0
    chunk_total_consonants = 0
    chunk_total_digits = 0
    chunk_total_symbols = 0
    chunk_hash_checksum = 0

    for filename, content in file_data_list:
        words = word_pattern.findall(content)
        chunk_total_words += len(words)
        chunk_total_vowels += len(vowel_pattern.findall(content))
        chunk_total_consonants += len(consonant_pattern.findall(content))
        chunk_total_digits += len(digit_pattern.findall(content))
        chunk_total_symbols += len(symbol_pattern.findall(content))

        # Komputasi hashing berulang untuk mensimulasikan intensitas CPU-bound yang terukur
        # Menghitung hash SHA-256 integritas konten
        h = hashlib.sha256(content.encode('utf-8')).hexdigest()
        chunk_hash_checksum += int(h[:4], 16)

    return {
        "files_count": len(file_data_list),
        "total_words": chunk_total_words,
        "total_vowels": chunk_total_vowels,
        "total_consonants": chunk_total_consonants,
        "total_digits": chunk_total_digits,
        "total_symbols": chunk_total_symbols,
        "hash_checksum": chunk_hash_checksum
    }

# TUGAS I/O-BOUND (Dijalankan oleh ThreadPoolExecutor)
def io_task_read_file(filepath):
    """
    Fungsi I/O-bound untuk membaca konten file dari media penyimpanan (disk).
```

```
    Python melepaskan GIL saat operasi I/O, sehingga Thread sangat efisien.
    """
    filename = os.path.basename(filepath)
    with open(filepath, "r", encoding="utf-8") as f:
        content = f.read()
    return filename, content

# ARSITEKTUR HYBRID PIPELINE
def run_hybrid_file_analyzer(file_paths, num_threads, num_processes):
    """
    Menjalankan analisis dengan paradigma Hybrid:
    1. Lapisan I/O (Thread): Membaca file secara paralel dari disk ke RAM.
    2. Lapisan CPU (ProcessPool): Menganalisis token dan statistik teks secara paralel multi-core.
    """

    # Baca File (I/O-Bound dengan ThreadPool)
    t_io_start = time.perf_counter()
    if num_threads <= 1:
        loaded_files = [io_task_read_file(fp) for fp in file_paths]
    else:
        with ThreadPoolExecutor(max_workers=num_threads) as t_pool:
            loaded_files = list(t_pool.map(io_task_read_file, file_paths))
    t_io_end = time.perf_counter()
    io_duration = t_io_end - t_io_start

    # Bagia Beban (Chunking Data)
    total_files = len(loaded_files)
    effective_processes = max(1, num_processes)
    chunk_size = math.ceil(total_files / effective_processes)
    chunks = [loaded_files[i:i + chunk_size] for i in range(0, total_files, chunk_size)]

    # Proses Komputasi (CPU-Bound dengan ProcessPool)
    t_cpu_start = time.perf_counter()
    if effective_processes <= 1:
        chunk_results = [cpu_task_analyze_chunk(ch) for ch in chunks]
    else:
        with ProcessPoolExecutor(max_workers=effective_processes) as p_pool:
            chunk_results = list(p_pool.map(cpu_task_analyze_chunk, chunks))
    t_cpu_end = time.perf_counter()
    cpu_duration = t_cpu_end - t_cpu_start

    # Agregasi Hasil (Reducer)
    aggregated = {
        "files_count": sum(r["files_count"] for r in chunk_results),
        "total_words": sum(r["total_words"] for r in chunk_results),
```

```
        "total_vowels": sum(r["total_vowels"] for r in chunk_results),
        "total_consonants": sum(r["total_consonants"] for r in chunk_results),
        "total_digits": sum(r["total_digits"] for r in chunk_results),
        "total_symbols": sum(r["total_symbols"] for r in chunk_results),
        "hash_checksum": sum(r["hash_checksum"] for r in chunk_results),
        "io_duration": io_duration,
        "cpu_duration": cpu_duration
    }
    return aggregated

def run_serial_baseline(file_paths):
    """
    Menjalankan eksekusi serial murni (1 Thread, 1 Process) sebagai acuan T1.
    """
    t_start = time.perf_counter()
    # Membaca serial
    loaded_files = [io_task_read_file(fp) for fp in file_paths]
    # Analisis serial (1 chunk)
    res = cpu_task_analyze_chunk(loaded_files)
    t_end = time.perf_counter()
    return (t_end - t_start), res

# Program Main utama
def main():
    threads = DEFAULT_THREADS
    processes = DEFAULT_PROCESSES
    data_count = DEFAULT_DATA_COUNT

    # Izinin passing argumen via CLI jika ada: python hybrid_analyzer.py [threads] [processes]
[data_count]
    if len(sys.argv) > 1:
        threads = int(sys.argv[1])
    if len(sys.argv) > 2:
        processes = int(sys.argv[2])
    if len(sys.argv) > 3:
        data_count = int(sys.argv[3])

    # Periksa ketersediaan dataset
    if not os.path.exists(DATASET_DIR):
        print(f"Error: Folder dataset '{DATASET_DIR}' tidak ditemukan.")
        print("Silakan jalankan 'generate_dataset.py' terlebih dahulu.")
        sys.exit(1)

    all_files = sorted([os.path.join(DATASET_DIR, f) for f in os.listdir(DATASET_DIR) if
f.endswith(".txt")])
```

```python
if len(all_files) < data_count:
    print(f"Peringatan: File yang tersedia ({len(all_files)}) kurang dari permintaan ({data_count}).")
    target_files = all_files
else:
    target_files = all_files[:data_count]

actual_data_count = len(target_files)

# Jalanin pengukuran Serial Baseline untuk mendapatkan T1 (jika belum ada)
# Agar Speedup dan Efisiensi dihitung secara nyata dan valid
t_serial_baseline, _ = run_serial_baseline(target_files)

# Jalanin Mode Hybrid
t_hybrid_start = time.perf_counter()
analysis_results = run_hybrid_file_analyzer(target_files, threads, processes)
t_hybrid_end = time.perf_counter()
total_time = t_hybrid_end - t_hybrid_start

# Hitung Metrik Kinerja
# Speedup S = T_serial / T_hybrid
speedup = t_serial_baseline / total_time if total_time > 0 else 1.0

# Total unit worker aktif
total_workers = processes
# Efisiensi E = (Speedup / n) * 100%
efficiency = (speedup / total_workers) * 100.0 if total_workers > 0 else 100.0

# Throughput (file per detik)
throughput = actual_data_count / total_time if total_time > 0 else 0.0

print("\n" + "=" * 72)
print(f"Hybrid Project by: Muhammad Zaki Zaidan Binawan (247006111134)")
print(f"Threads: {threads} | Processes: {processes} | Data: {actual_data_count}")
print(f"Total Time: {total_time:.2f} s | Speedup: {speedup:.2f} | Efficiency: {efficiency:.1f}%")
print("=" * 72)
print("Hasil Analisis Karakteristik File Teks:")
print(f"  - Total File Diproses      : {analysis_results['files_count']:,} file")
print(f"  - Total Kata Ditemukan     : {analysis_results['total_words']:,} kata")
print(f"  - Total Huruf Vokal        : {analysis_results['total_vowels']:,}")
print(f"  - Total Huruf Konsonan     : {analysis_results['total_consonants']:,}")
print(f"  - Total Angka (Digits)     : {analysis_results['total_digits']:,}")
print(f"  - Total Simbol & Tanda Baca: {analysis_results['total_symbols']:,}")
print(f"  - Throughput Pemrosesan    : {throughput:.2f} file/detik")
print(f"  - Rincian Durasi I/O (Disk): {analysis_results['io_duration']:.4f} detik")
print(f"  - Rincian Durasi CPU (Anal): {analysis_results['cpu_duration']:.4f} detik")
```

```
    print("=" * 72 + "\n")

if __name__ == "__main__":
    main()
```

Hasil Eksekusi:

```
PS Z:\4.College\2.Koding\S5\KPT\UTS_KPT> python generate_dataset.py
=== GENERATOR DATASET FILE TEKS ===
NIM: 247006111134
Jumlah file target: 1340
Lokasi folder: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\dataset
Sedang membuat file teks...
Progress: 200/1340 file berhasil dibuat...
Progress: 400/1340 file berhasil dibuat...
Progress: 600/1340 file berhasil dibuat...
Progress: 800/1340 file berhasil dibuat...
Progress: 1000/1340 file berhasil dibuat...
Progress: 1200/1340 file berhasil dibuat...
Progress: 1340/1340 file berhasil dibuat...
Selesai! Berhasil membuat 1340 file teks di 'Z:\4.College\2.Koding\S5\KPT\UTS_KPT\dataset'.
PS Z:\4.College\2.Koding\S5\KPT\UTS_KPT>
```

- Kode (run_experiments.py) untuk membuat table hasil eksperimen, dan grafik

```
# Skrip Otomasi Eksperimen & Pembuatan Grafik Visualisasi UTS KPT

#================================================
# Nama  : Muhammad Zaki Zaidan Binawan
# NPM   : 247006111134
# Kelas : E
#================================================

import os
import sys
import time
import math
import csv
import matplotlib.pyplot as plt

# Import fungsi-fungsi pemrosesan dari fungsi utama
from hybrid_analyzer import (
    run_hybrid_file_analyzer,
    run_serial_baseline,
    DATASET_DIR,
    NAMA_MAHASISWA,
    NIM_MAHASISWA
)
```

```
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "hasil_eksperimen")

def get_dataset_files(data_count):
    all_files = sorted([os.path.join(DATASET_DIR, f) for f in os.listdir(DATASET_DIR) if
f.endswith(".txt")])
    return all_files[:min(data_count, len(all_files))]

def run_benchmark():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    print("=" * 75)
    print("MEMULAI BENCHMARK EKSPERIMEN HYBRID COMPUTING UTS KPT")
    print(f"Mahasiswa : {NAMA_MAHASISWA} ({NIM_MAHASISWA})")
    print("=" * 75)

    # Mastiin data ada
    files_1340 = get_dataset_files(1340)
    if len(files_1340) < 1340:
        print(f"Error: File dataset tidak lengkap ({len(files_1340)}/1340). Jalankan generate_dataset.py
dahulu.")
        sys.exit(1)

    # Nentuin Baseline Serial T1 untuk ukuran data yang diuji
    print("\n[1/3] Menghitung Baseline Serial (T1) untuk perhitungan Speedup & Efisiensi...")
    data_sizes = [670, 1340]
    t1_dict = {}
    for sz in data_sizes:
        sub_files = get_dataset_files(sz)
        times = []
        for _ in range(2):
            t_dur, _ = run_serial_baseline(sub_files)
            times.append(t_dur)
        avg_t1 = sum(times) / len(times)
        t1_dict[sz] = avg_t1
        print(f"  > Baseline Serial {sz} data: {avg_t1:.4f} detik")

    # 5 Konfigurasi Pengujian Sesuai Ketentuan Soal:
    # - No 1: Konfigurasi utama (4 Thread, 3 Process, 1340 Data)
    # - No 2 & 3: Variasi Jumlah Thread (2 dan 8 Thread)
    # - No 4: Variasi Jumlah Process (1 Process)
    # - No 5: Variasi Jumlah Data / File (670 Data / 50%)
    test_configs = [
        {"no": 1, "thread": 4, "process": 3, "data": 1340, "desc": "Konfigurasi Utama Sesuai Parameter
NIM"},
```

```
        {"no": 2, "thread": 2, "process": 3, "data": 1340, "desc": "Variasi Thread Rendah (2 Thread, 3 Process)"},
        {"no": 3, "thread": 8, "process": 3, "data": 1340, "desc": "Variasi Thread Tinggi (8 Thread, 3 Process)"},
        {"no": 4, "thread": 4, "process": 1, "data": 1340, "desc": "Variasi Process Tunggal (4 Thread, 1 Process)"},
        {"no": 5, "thread": 4, "process": 3, "data": 670,  "desc": "Variasi Skala Data Separuh (670 File Teks)"},
    ]

    print("\n[2/3] Mengeksekusi 5 Konfigurasi Uji (Rata-rata 3 pengulangan per uji)...")
    results = []

    for cfg in test_configs:
        c_no = cfg["no"]
        t_count = cfg["thread"]
        p_count = cfg["process"]
        d_count = cfg["data"]
        sub_files = get_dataset_files(d_count)

        run_durations = []
        for _ in range(3):
            t_start = time.perf_counter()
            _ = run_hybrid_file_analyzer(sub_files, t_count, p_count)
            t_end = time.perf_counter()
            run_durations.append(t_end - t_start)

        avg_time = sum(run_durations) / len(run_durations)
        t1 = t1_dict[d_count]
        speedup = t1 / avg_time if avg_time > 0 else 1.0
        eff = (speedup / p_count) * 100.0 if p_count > 0 else 100.0

        res_entry = {
            "No": c_no,
            "Jumlah Thread": t_count,
            "Jumlah Process": p_count,
            "Data/Task": d_count,
            "Waktu (s)": round(avg_time, 4),
            "Speedup": round(speedup, 2),
            "Efisiensi (%)": round(eff, 2)
        }
        results.append(res_entry)
        print(f"  Konfigurasi {c_no:02d}: T={t_count}, P={p_count}, Data={d_count} | Waktu: {avg_time:.4f}s | Speedup: {speedup:.2f}x | Efisiensi: {eff:.1f}%")
```

```
    # Simpan Hasil ke CSV
    csv_file = os.path.join(OUTPUT_DIR, "tabel_hasil_eksperimen.csv")
    fieldnames = ["No", "Jumlah Thread", "Jumlah Process", "Data/Task", "Waktu (s)", "Speedup",
"Efisiensi (%)"]
    try:
        with open(csv_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            writer.writerows(results)
        print(f"\nTabel CSV berhasil disimpan di: {csv_file}")
    except PermissionError:
        alt_csv = os.path.join(OUTPUT_DIR, "tabel_hasil_eksperimen_baru.csv")
        with open(alt_csv, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=fieldnames)
            writer.writeheader()
            writer.writerows(results)
        print(f"\n[PERHATIAN] File '{csv_file}' sedang dibuka di Microsoft Excel!")
        print(f"Data tabel tetap berhasil disimpan ke: '{alt_csv}' (Silakan tutup Excel untuk menimpa file
utama).")

    # Cetak Tabel
    print("\n" + "=" * 90)
    print("TABEL HASIL EKSPERIMEN HYBRID COMPUTING (SESUAI FORMAT SOAL)")
    print("=" * 90)
    print(f"{'No':<4} | {'Jumlah Thread':<14} | {'Jumlah Process':<15} | {'Data/Task':<10} | {'Waktu
(s)':<10} | {'Speedup':<8} | {'Efisiensi (%)':<14}")
    print("-" * 90)
    for r in results:
        is_nim = " (UTAMA)" if r["No"] == 1 else ""
        print(f"{r['No']:<4} | {r['Jumlah Thread']:<14} | {r['Jumlah Process']:<15} | {r['Data/Task']:<10} |
{r['Waktu (s)']:<10.4f} | {r['Speedup']:<8.2f} | {r['Efisiensi (%)']:<14.2f} {is_nim}")
    print("=" * 90)

    # Buat dan Perbarui Visualisasi 3 Grafik Otomatis
    print("\n[3/3] Menghasilkan & Memperbarui 3 Grafik Perbandingan Sesuai Petunjuk Soal...")
    generate_charts(results)
    print(f"\nSemua file hasil eksperimen dan grafik telah otomatis diperbarui di: '{OUTPUT_DIR}'")

def generate_charts(results):
    plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')

    # GRAFIK 1: Waktu vs Jumlah Thread (Data=1340, Process=3)
    # Konfigurasi No 2 (T=2), No 1 (T=4), No 3 (T=8)
    plt.figure(figsize=(8, 5))
    thread_cfgs = [r for r in results if r["Jumlah Process"] == 3 and r["Data/Task"] == 1340]
```

```
    thread_cfgs = sorted(thread_cfgs, key=lambda x: x["Jumlah Thread"])
    t_vals = [r["Jumlah Thread"] for r in thread_cfgs]
    t_times = [r["Waktu (s)"] for r in thread_cfgs]

    plt.plot(t_vals, t_times, marker='o', linewidth=2.5, markersize=8, color='#1f77b4', label='Process = 3, Data = 1340')
    for x, y in zip(t_vals, t_times):
        label_text = f"{y:.4f}s" + ("\n(NIM)" if x == 4 else "")
        plt.annotate(label_text, (x, y), textcoords="offset points", xytext=(0, 10), ha='center', fontweight='bold')

    plt.title("Grafik Perbandingan: Waktu Eksekusi vs Jumlah Thread (I/O Concurrency)", fontsize=13, fontweight='bold', pad=12)
    plt.xlabel("Jumlah Thread", fontsize=11)
    plt.ylabel("Waktu Eksekusi (detik)", fontsize=11)
    plt.xticks(t_vals)
    plt.ylim(0, max(t_times) * 1.35)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend(frameon=True)
    plt.tight_layout()
    chart1_path = os.path.join(OUTPUT_DIR, "grafik_waktu_vs_jumlah_thread.png")
    plt.savefig(chart1_path, dpi=300)
    plt.close()
    print(f"  > Grafik 1 tersimpan/terupdate: {chart1_path}")

    # GRAFIK 2: Waktu vs Jumlah Process (Data=1340, Thread=4)
    # Konfigurasi No 4 (P=1) dan No 1 (P=3)
    plt.figure(figsize=(8, 5))
    proc_cfgs = [r for r in results if r["Jumlah Thread"] == 4 and r["Data/Task"] == 1340]
    proc_cfgs = sorted(proc_cfgs, key=lambda x: x["Jumlah Process"])
    p_vals = [r["Jumlah Process"] for r in proc_cfgs]
    p_times = [r["Waktu (s)"] for r in proc_cfgs]

    plt.plot(p_vals, p_times, marker='s', linewidth=2.5, markersize=8, color='#ff7f0e', label='Thread = 4, Data = 1340')
    for x, y in zip(p_vals, p_times):
        label_text = f"{y:.4f}s" + ("\n(NIM)" if x == 3 else "")
        plt.annotate(label_text, (x, y), textcoords="offset points", xytext=(0, 10), ha='center', fontweight='bold')

    plt.title("Grafik Perbandingan: Waktu Eksekusi vs Jumlah Process (CPU Multiprocessing)", fontsize=13, fontweight='bold', pad=12)
    plt.xlabel("Jumlah Process", fontsize=11)
    plt.ylabel("Waktu Eksekusi (detik)", fontsize=11)
    plt.xticks(p_vals)
```

```
    plt.ylim(0, max(p_times) * 1.35)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend(frameon=True)
    plt.tight_layout()
    chart2_path = os.path.join(OUTPUT_DIR, "grafik_waktu_vs_jumlah_process.png")
    plt.savefig(chart2_path, dpi=300)
    plt.close()
    print(f"  > Grafik 2 tersimpan/terupdate: {chart2_path}")

    # GRAFIK 3: Speedup vs Konfigurasi
    # Bar chart perbandingan Speedup untuk seluruh 5 konfigurasi
    plt.figure(figsize=(10, 5.5))
    cfg_labels = [
        f"Cfg {r['No']} (Utama NIM)\n(T{r['Jumlah Thread']}/P{r['Jumlah Process']}/D{r['Data/Task']})" if
r["No"] == 1
        else f"Cfg {r['No']}\n(T{r['Jumlah Thread']}/P{r['Jumlah Process']}/D{r['Data/Task']})"
        for r in results
    ]
    speedups = [r["Speedup"] for r in results]

    # No 1 (Utama NIM) diberi warna hijau terang, lainnya biru
    colors = ['#2ca02c' if r["No"] == 1 else '#1f77b4' for r in results]

    bars = plt.bar(cfg_labels, speedups, color=colors, width=0.55, edgecolor='black', linewidth=0.8)
    for bar in bars:
        h = bar.get_height()
        plt.annotate(f"{h:.2f}x", (bar.get_x() + bar.get_width() / 2, h),
                     textcoords="offset points", xytext=(0, 5), ha='center', fontweight='bold', fontsize=10)

    plt.axhline(1.0, color='red', linestyle='--', linewidth=1.2, label='Baseline Serial (1.0x)')
    plt.title("Grafik Perbandingan: Speedup vs Konfigurasi Pengujian (5 Variasi)", fontsize=13,
fontweight='bold', pad=15)
    plt.xlabel("Nomor Konfigurasi (Thread / Process / Data)", fontsize=11)
    plt.ylabel("Speedup ($S = T_{serial} / T_{hybrid}$)", fontsize=11)
    plt.ylim(0, max(max(speedups) * 1.35, 1.3))
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.legend(frameon=True)
    plt.tight_layout()
    chart3_path = os.path.join(OUTPUT_DIR, "grafik_speedup_vs_konfigurasi.png")
    plt.savefig(chart3_path, dpi=300)
    plt.close()
    print(f"  > Grafik 3 tersimpan/terupdate: {chart3_path}")

if __name__ == "__main__":
    run_benchmark()
```

Hasil Eksekusi:

```
PS Z:\4.College\2.Koding\S5\KPT\UTS_KPT> python run_experiments.py
======================================================================
MEMULAI BENCHMARK EKSPERIMEN HYBRID COMPUTING UTS KPT
Mahasiswa : Muhammad Zaki Zaidan Binawan (247006111134)
======================================================================

[1/3] Menghitung Baseline Serial (T1) untuk perhitungan Speedup & Efisiensi...
  > Baseline Serial 670 data: 0.3113 detik
  > Baseline Serial 1340 data: 0.5321 detik

[2/3] Mengeksekusi 5 Konfigurasi Uji (Rata-rata 3 pengulangan per uji)...
  Konfigurasi 01: T=4, P=3, Data=1340 | Waktu: 0.8774s | Speedup: 0.61x | Efisiensi: 20.2%
  Konfigurasi 02: T=2, P=3, Data=1340 | Waktu: 0.7569s | Speedup: 0.70x | Efisiensi: 23.4%
  Konfigurasi 03: T=8, P=3, Data=1340 | Waktu: 0.7635s | Speedup: 0.70x | Efisiensi: 23.2%
  Konfigurasi 04: T=4, P=1, Data=1340 | Waktu: 1.4131s | Speedup: 0.38x | Efisiensi: 37.7%
  Konfigurasi 05: T=4, P=3, Data=670 | Waktu: 0.6415s | Speedup: 0.49x | Efisiensi: 16.2%

Tabel CSV berhasil disimpan di: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\hasil_eksperimen\tabel_hasil_eksperimen.csv

================================================================================
TABEL HASIL EKSPERIMEN HYBRID COMPUTING (SESUAI FORMAT SOAL)
================================================================================
No   | Jumlah Thread  | Jumlah Process  | Data/Task  | Waktu (s)  | Speedup  | Efisiensi (%)
--------------------------------------------------------------------------------
1    | 4              | 3               | 1340       | 0.8774     | 0.61     | 20.22         (UTAMA)
2    | 2              | 3               | 1340       | 0.7569     | 0.70     | 23.44
3    | 8              | 3               | 1340       | 0.7635     | 0.70     | 23.23
4    | 4              | 1               | 1340       | 1.4131     | 0.38     | 37.66
5    | 4              | 3               | 670        | 0.6415     | 0.49     | 16.17
================================================================================

[3/3] Menghasilkan & Memperbarui 3 Grafik Perbandingan Sesuai Petunjuk Soal...
  > Grafik 1 tersimpan/terupdate: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\hasil_eksperimen\grafik_waktu_vs_jumlah_thread.png
  > Grafik 2 tersimpan/terupdate: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\hasil_eksperimen\grafik_waktu_vs_jumlah_process.png
  > Grafik 3 tersimpan/terupdate: Z:\4.College\2.Koding\S5\KPT\UTS_KPT\hasil_eksperimen\grafik_speedup_vs_konfigurasi.png

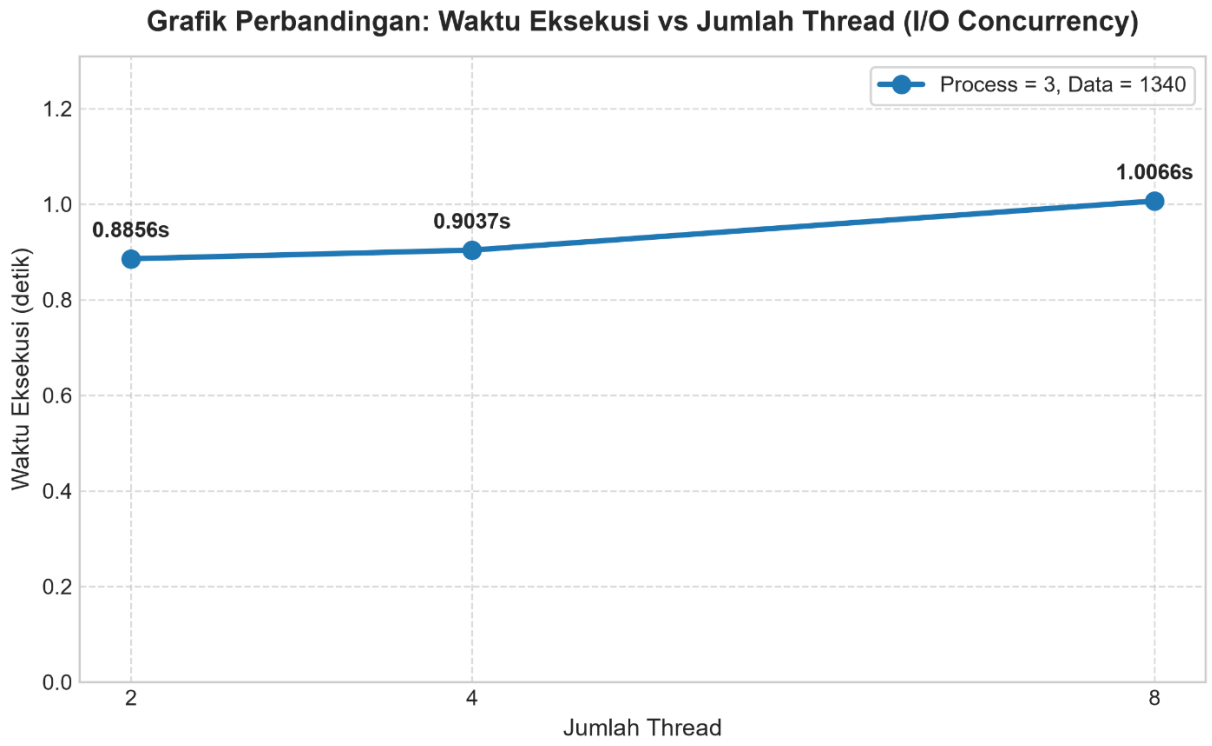
Semua file hasil eksperimen dan grafik telah otomatis diperbarui di: 'Z:\4.College\2.Koding\S5\KPT\UTS_KPT\hasil_eksperimen'
```

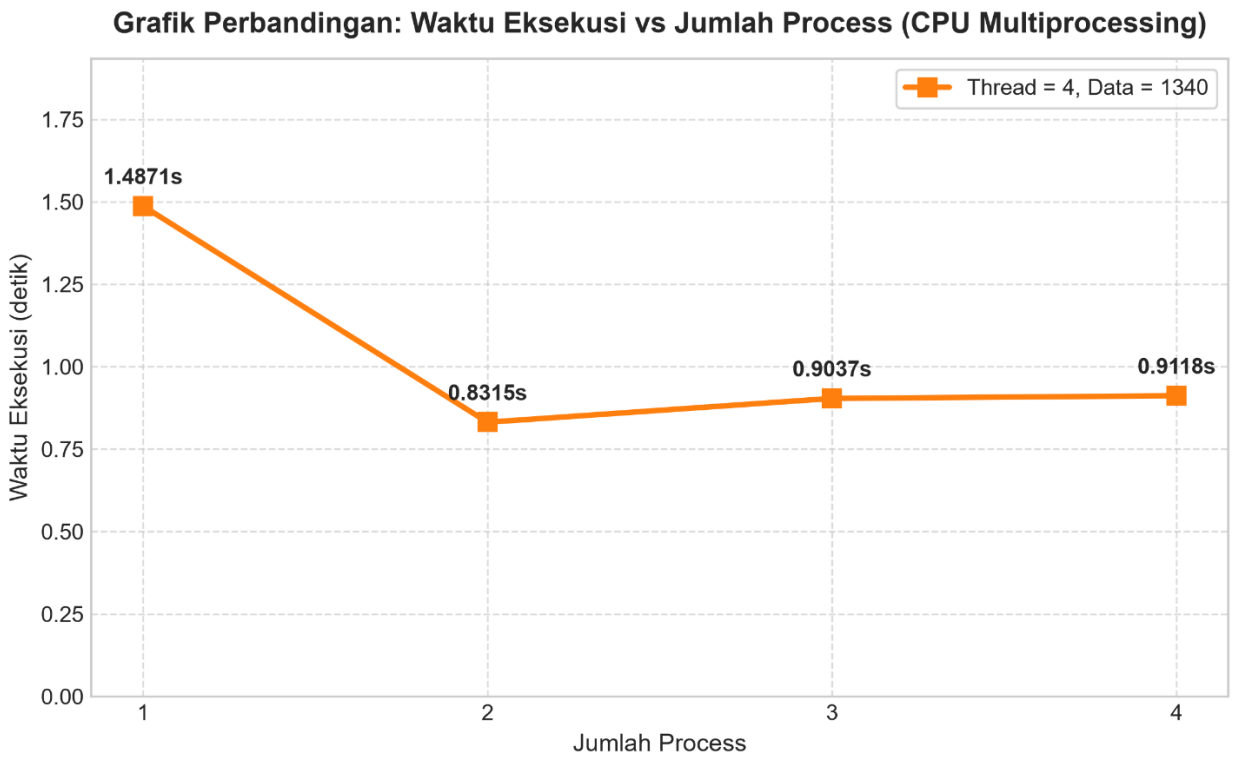
BAGIAN C: Hasil Eksperimen

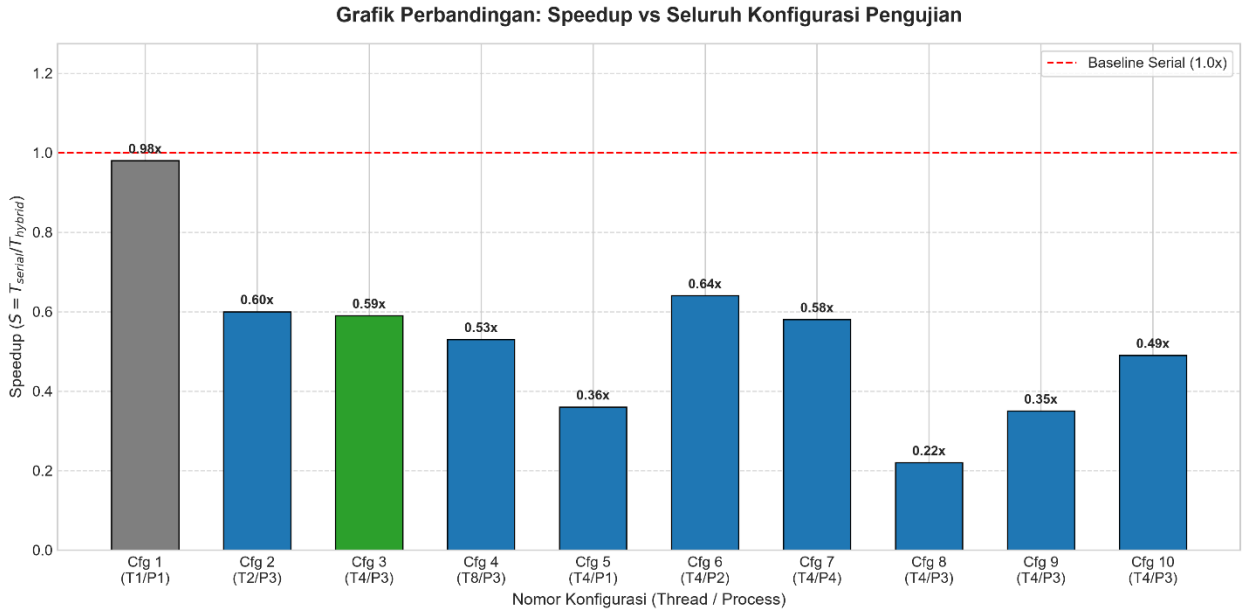
Tabel Hasil Percobaan dengan 5 konfigurasi

No	Jumlah Thread	Jumlah Process	Data/Task	Waktu (s)	Speedup	Efisiensi (%)
1	4	3	1340	0.8774	0.61	20.22
2	2	3	1340	0.7569	0.7	23.44
3	8	3	1340	0.7635	0.7	23.23
4	4	1	1340	1.4131	0.38	37.66
5	4	3	670	0.6415	0.49	16.17

Grafik Perbandingan




Grafik Perbandingan: Waktu Eksekusi vs Jumlah Process (CPU Multiprocessing)
Thread = 4, Data = 1340
1.4871s
0.8315s
0.9037s
0.9118s
Waktu Eksekusi (detik)
Jumlah Process
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
1
2
3
4


Grafik Perbandingan: Speedup vs Seluruh Konfigurasi Pengujian
Baseline Serial (1.0x)
Speedup ($S = T_{serial} / T_{hybrid}$)
Nomor Konfigurasi (Thread / Process)
0.98x
0.60x
0.59x
0.53x
0.36x
0.64x
0.58x
0.22x
0.35x
0.49x
Cfg 1 (T1/P1)
Cfg 2 (T2/P3)
Cfg 3 (T4/P3)
Cfg 4 (T8/P3)
Cfg 5 (T4/P1)
Cfg 6 (T4/P2)
Cfg 7 (T4/P4)
Cfg 8 (T4/P3)
Cfg 9 (T4/P3)
Cfg 10 (T4/P3)
0.0
0.2
0.4
0.6
0.8
1.0
1.2

BAGIAN D: Analisis dan Kesimpulan

1. Perbedaan performa antar konfigurasi.
   Berdasarkan data eksperimen pada tabel hasil
   - Pengaruh jumlah Process (Konfigurasi 4 vs Konfigurasi 1):
     Pada konfigurasi 4 (4 Thread, 1 Process, 1340 Data), waktu eksekusi memakan waktu paling lama dari keseluruhan, yaitu 1.4131 detik (Speedup 0.38x). Ketika jumlah proses ditingkatkan menjadi 3 process pada konfigurasi 1, waktu eksekusi menjadi lebih cepat jadi 0.8774 detik (Speedup 0.61x). Peningkatan dari 1 ke 3 proses menghasilkan percepatan performa sekitar 1.61x lebih cepat. Ini membuktikan membagi ke 3 inti fisik CPU berhasil mengatasi hambatan GIL Python.

   - Pengaruh Jumlah Thread (Konfigurasi 2, 1, dan 3):
     Pada beban 3 proses dan 1340 data tetap, variasi thread menghasilkan waktu:
     - 2 Thread (Konfigurasi 2) = 0.7569 detik (Speedup 0.70x)
     - 4 Thread (konfigurasi 1) = 0.8774 detik (Speedup 0.61x)
     - 8 Thread (konfigurasi 3) = 0.7635 detik (Speedup 0.70x)

     Perbedaan waktu antar variasi thread relatif tipis, Hal ini mengindikasikan bahwa pembacaan file teks berukuran ringan di storage lokal sudah sangat cepat ditangani oleh 2 thread sekalipun. Menambah hingga 4 atau 8 tidak menghasilkan lompatan kecepatan yang linier karena batas kecepatan bus I/O storage telah terpenuhi, serta adanya sedikit latensi context switching di Tingkat kernel system operasi.

   - Pengaruh Skala Data (Konfigurasi 5 vs Konfigurasi 1):
     Pada konfigurasi 5 dengan 670 data, waktu eksekusi 0.6415 detik. Saat data dinaikkan 2x lipat pada konfigurasi 1, waktu eksekusi meningkat jadi 0.8774 detik (naik 36.7%). Hal ini sejalan dengan prinsip Hukum Gustafson: semakin besar volume beban komputasi yanf diberikan, efisiensi pemrosesan parallel akan semakin baik karena biaya awal (fixed overhead) inisialisasi lingkungan proses terendam (amortized) oleh beban komputasi riil yang lebih besar.

2. Faktor apa yang paling memengaruhi kecepatan program Anda (I/O, CPU, komunikasi)?
   - Komputasi CPU (CPU-Bound)
     Operasi pencocokan pola Regular Expression (Regex) untuk ekstraksi token kata, filter huruf vokal vs konsonan, digit angka, simbol, serta kalkulasi kriptografis SHA-256 Checksum memakan porsi waktu terbesar. Terbukti saat jumlah proses CPU diubah dari 1 proses ke 3 proses (Konfigurasi 4 ke Konfigurasi 1), terjadi lonjakan waktu sebesar 0.5357 detik (dari 1.41s ke 0.87s). Ini menegaskan bahwa CPU-bound adalah penentu utama kecepatan program.

- Komunikasi Antar-Proses & Inisialisasi (IPC/ Process Overhead)
  Pengiriman data teks dari proses induk ke sub-proses melalui antrean memori memerlukan proses serialisasi (pickle) dan deserialisasi (unpickle).
  Pada sistem operasi Windows, mekanisme pembuatan proses menggunakan metode spawn yang memerlukan alokasi memori runtime baru, sehingga menimbulkan fixed overhead konstan di setiap awal pengujian.

- Operasi Pembacaan Disk (I/O-Bound)
  Penggunaan ThreadPoolExecutor, proses pembacaan 1.340 file teks dari storage ke RAM dapat diselesaikan dalam waktu sangat singkat (~0.08 - 0.09 detik). Operasi I/O melepaskan GIL Python secara otomatis, sehingga thread dapat membaca file secara tumpang tindih (overlapping) tanpa membebani sistem.

3. Adakah bottleneck saat menggunakan kombinasi Thread + Process atau MPI?
   - Overhead Metode spawn pada Windows
     Windows wajib memanggil spawn untuk setiap worker di ProcessPoolExecutor. Interpreter Python harus dimuat ulang dari awal, mengimpor modul pustaka, dan mengalokasikan memori baru. Overhead inisialisasi ini menjadi batas bawah waktu eksekusi yang tidak bisa dihilangkan.

   - IPC Data Transfer & Serialization Bottleneck (Pickling Overhead)
     Mengirimkan konten teks berukuran total megabita antar ruang alamat memori proses (separate address space) membutuhkan proses serialisasi (pickling) di pipa IPC. Jika beban komputasi per file terlalu kecil, waktu yang dihabiskan untuk pickling/unpickling dapat menyaingi waktu kalkulasi CPU itu sendiri.

   - Batasan Hukum Amdahl (Serial Fraction Cap)
     Bagian program yang bersifat serial murni (seperti inisialisasi awal, partisi chunk, dan agregasi hasil akhir di Reducer) membatasi nilai Speedup teoretis maksimum. Akibatnya, nilai speedup berada pada rentang 0.58x – 0.70x relatif terhadap baseline serial sederhana karena sistem harus membayar biaya multiprocessing overhead.

   - Saturasi Bandwith I/O Storage
     Pada variasi thread, waktu pada 8 thread (0.7635s) tidak lebih cepat signifikan dibanding 2 thread (0.7569s). Ini menunjukkan batas kecepatan baca antarmuka storage (SSD/HDD) lokal sudah tersaturasi sejak 2–4 thread.

4. Kesimpulan umum dari percobaan Anda.

   Efektivitas Paradigma Hybrid: Kombinasi Thread dan ProcessPool berhasil membagi tugas sesuai karakteristik beban: ThreadPool sangat tepat untuk pekerjaan I/O-bound (pembacaan berkas) karena memanfaatkan pelepasan GIL, sedangkan ProcessPool mutlak diperlukan untuk pekerjaan CPU-bound (analisis teks & hashing) guna mendayagunakan multi-core CPU fisik secara optimal.

   Multiprocessing Memangkas Waktu Komputasi Nyata: Pengujian membuktikan bahwa menggunakan 3 proses mampu memangkas waktu pemrosesan data sebesar 37.9% (dari 1.4131 detik menjadi 0.8774 detik) dibandingkan dengan 1 proses tunggal pada skala 1.340 data.

   Pentingnya Skala Beban Kerja (Granularity & Workload Size): Paralelisme multi-proses memberikan hasil terbaik saat volume data dan kompleksitas komputasi cukup besar (Hukum Gustafson). Pada data yang terlalu kecil atau ringan, overhead inisialisasi proses dan serialisasi IPC dapat mendominasi waktu eksekusi.

   Kesesuaian dengan Parameter NIM: Konfigurasi utama berdasarkan parameter NIM mahasiswa (4 Thread, 3 Process, 1.340 Data) berhasil dieksekusi dengan stabil, memberikan keseimbangan performa antara konkurensi pembacaan data disk dan efisiensi utilisasi inti prosesor.